Student Task Breakdown — Week 9 (Integration)

Scope

Duration: Mar 9–13, 2026 (1 week)
Budget: 8 hrs/week per student (24hrs total)
Goal: A full prototype! Four screens functional, calling the real Flask API. By Friday, you can type a sentence, select a token, and see its trajectory, the attention patterns, and the logit lens predictions in the browser.

Supervision cadence

- **Monday AM:** Send task briefs
- **Friday:** Deliverable review (full click-through flow)

Mario — Frontend Scaffold + `/tokenize` (8 hrs)

Task 9.1: Create the frontend file structure (~1 hr)

What to do:

- Create a `frontend/` folder at the root of the project (same level as `backend/`)
- Create three files:
  - `frontend/index.html` — all screens live here as hidden `<div>` sections
  - `frontend/style.css` — paste in Polly's CSS design tokens when she sends them
  - `frontend/app.js` — all JavaScript
- In `index.html`, scaffold the four screens as hidden divs with IDs:

```
<div id="screen-input" class="screen active">
<div id="screen-tokens" class="screen">
<div id="screen-trace" class="screen">
</div>
<div id="screen-attention" class="screen">
</div>
<div id="screen-predict" class="screen">
```

- Add a helper function in `app.js` to switch between screens:

```
function showScreen(id) {
  document.querySelectorAll('.screen').forEach(s =>
s.classList.remove('active'));
  document.getElementById(id).classList.add('active');
}
```

Done when: Opening `index.html` in a browser shows one screen at a time. No styling needed yet.

Task 9.2: Wire `/tokenize` → token chip display (~4 hrs)

What to do:

- In `screen-input`, add: a `<textarea>` pre-filled with `"The cat sat on the mat"` and a `Tokenize` button
- In `app.js`, write the fetch function:

```
async function tokenize(text) {
  try {
    const res = await fetch('http://localhost:5001/tokenize', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text })
    });
    return res.json();
  } catch {
    // fallback to example JSON when server is offline
    const res = await fetch('../backend/examples/example1_tokenize.json');
    return res.json();
  }
}
```

- When `Tokenize` is clicked:
  1. Call `tokenize(text)`
  2. Render each token as a `<span class="token-chip">` inside `screen-tokens`
  3. The `<|endoftext|>` chip gets class `token-bos` and is not clickable
  4. All other chips are clickable — clicking toggles class `selected` (max 3 selected at once)

5. Show a `Trace` button that activates once at least 1 token is selected
6. Switch to `screen-tokens`

- Store the full token list and selected indices in variables at the top of `app.js`:

```
let currentText = '';
let currentTokens = [];
let selectedIndices = new Set();
```

Done when: Type a sentence, click Tokenize, see token chips appear including `<|endoftext|>`. Click chips to select/deselect them (they highlight). Trace button appears when at least one is selected.

Reference: Polly's mockup — screen 3 (token chip row with "The" highlighted in blue).

Task 9.3: Fix `API.md` with real curl output (~2 hrs)

What to do:

- Start the Flask server (`python app.py`)
- Run each endpoint with `"The cat sat on the mat"` using the exact `curl` commands in `API.md`
- Replace every example JSON block in `API.md` with the actual terminal output
- Update every `localhost:5000` → `localhost:5001`
- For `/attention`: show both the single-head case (`head` provided) and the all-heads case (`head` omitted, returns 3D array)

Done when: Copy-paste a `curl` command from `API.md`, run it, and the output matches the docs exactly.

Steven — Fetch Wiring: Trace, Attention, Predict (8 hrs)

Task 9.4: Write all four fetch functions (~1 hr)

What to do:

- In `app.js`, write fetch functions for the remaining three endpoints, each with a fallback to the example JSON files:

```javascript
async function trace(text, tokenIndices) {
  try {
    const res = await fetch('http://localhost:5001/trace', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text, token_indices: tokenIndices })
    });
    return res.json();
  } catch {
    const res = await fetch('../backend/examples/example1_trace.json');
    return res.json();
  }
}

async function attention(text, layer, head = null) {
  try {
    const body = head !== null ? { text, layer, head } : { text, layer };
    const res = await fetch('http://localhost:5001/attention', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
    return res.json();
  } catch {
    const res = await fetch('../backend/examples/example1_attention.json');
    return res.json();
  }
}

async function predict(text, tokenIndex) {
  try {
    const res = await fetch('http://localhost:5001/predict', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text, token_index: tokenIndex })
    });
    return res.json();
  } catch {
    const res = await fetch('../backend/examples/example1_predict.json');
    return res.json();
  }
}
```

Done when: Open the browser console, call each function manually with test data, get back valid JSON from the live server.

Task 9.5: Wire `/trace` → stacked planes (trajectory view) (~3 hrs)

What to do:

- When `Trace` is clicked in `screen-tokens`:
  1. Call `trace(currentText, [...selectedIndices])`
  2. Pass the response to `renderTrajectory(data)` (see below)
  3. Switch to `screen-trace`
- Write `renderTrajectory(data)`:
  - `data.trajectories` is an object: `{ "1": [{layer, x, y}, ...], "3": [...] }` — one array of 12 points per selected token
  - Normalize all x, y values across all tokens/layers to a 0–1 range (so dots stay within each plane's bounds)
  - For each layer (0–11), call `Polly's stacked planes component` passing the layer index and the dot positions for that layer
  - Dots for different tokens use different colors; opacity increases with layer number (lighter = early layers, darker = late layers)

```
function normalizeCoords(trajectories) {
  const allX = Object.values(trajectories).flat().map(p => p.x);
  const allY = Object.values(trajectories).flat().map(p => p.y);
  const minX = Math.min(...allX), maxX = Math.max(...allX);
  const minY = Math.min(...allY), maxY = Math.max(...allY);
  const norm = {};
  for (const [idx, points] of Object.entries(trajectories)) {
    norm[idx] = points.map(p => ({
      layer: p.layer,
      x: (p.x - minX) / (maxX - minX),
      y: (p.y - minY) / (maxY - minY)
    }));
  }
  return norm;
}
```

Done when: Click Trace with "The" selected → the stacked planes screen appears with a dot on each plane, positioned differently per layer. Different tokens show in different colors.

Task 9.6: Wire `/attention` → stacked planes (attention view) (~3 hrs)

What to do:

- Add a button or nav item to switch from the trajectory view to the attention view
- In `screen-attention`, add:
  - A head selector row (chips 0–11, same style as the layer/head selectors in Polly's mockup)
  - A toggle: `Single Head` / `All Layers`
- **Single head mode** (one heatmap, selected head):
  - When a head chip is clicked, call `attention(currentText, currentLayer, selectedHead)`
  - Pass `response.attention_matrix` to `renderHeatmap(matrix, planeElement)`
- **All-layers mode** (12 stacked planes, all heads averaged):
  - Call `attention(currentText, layer)` for all 12 layers in parallel:

```
const matrices = await Promise.all(
  Array.from({length: 12}, (_, layer) => attention(currentText, layer))
);
```

  - For each layer result, average the 12 heads client-side:

```
function averageHeads(allHeadsMatrix) {
  // allHeadsMatrix: [12 heads][n][n]
  return allHeadsMatrix[0].map((row, i) =>
    row.map((_, j) =>
      allHeadsMatrix.reduce((sum, head) => sum + head[i][j], 0) /
allHeadsMatrix.length
    )
  );
}
```

  - Pass the averaged matrix for each layer to `renderHeatmap(matrix, planeElement)`

Done when: Single head mode shows one heatmap that changes when you click a different head chip. All-layers mode shows 12 stacked planes each with a blue-gradient heatmap.

Polly — Design Tokens + Stacked Planes Component (8 hrs)

Task P9.1: Export CSS design tokens from Figma (~1 hr)

What to do:

- Open your Figma Specs page
- Create `frontend/tokens.css` with CSS custom properties for everything in the Specs page:

```
:root {
  /* Colors */
  --color-background: #f0f4ff;       /* the grid background */
  --color-plane: #ffffff;
  --color-plane-border: #000000;
  --color-token-selected: #c8dcff;   /* blue chip highlight */
  --color-attention-max: #3a7bd5;    /* darkest blue in heatmap */
  --color-token-1: /* your token color 1 */;
  --color-token-2: /* your token color 2 */;
  --color-token-3: /* your token color 3 */;

  /* Typography */
  --font-family: /* your font */;
  --font-size-label: /* */;
  --font-size-chip: /* */;
  --font-weight-bold: 700;

  /* Layout */
  --chip-border-radius: /* */;
  --chip-padding: /* */;
}
```

- Link `tokens.css` in `index.html` before `style.css`

Done when: Mario and Steven can use `var(--color-token-selected)` in their CSS and see the right blue.

Task P9.2: Build the stacked planes CSS component (~4 hrs)

What to do:
This is the core visual component. It is used for **both** the trajectory view and the attention view. The only difference is what gets drawn on each plane (a dot vs. a heatmap grid).

- In `style.css`, build the 3D stack using CSS perspective:

```
.plane-stack {
  perspective: 1200px;
  perspective-origin: 60% 40%;
```

```
  position: relative;
  width: 600px;
  height: 400px;
}

.plane {
  position: absolute;
  width: 100%;
  height: 100%;
  border: 2px solid var(--color-plane-border);
  background: var(--color-plane);
  transform-style: preserve-3d;
  /* each plane is offset in Z based on its layer index */
  /* JS will set --layer-index as a CSS variable on each element */
  transform: rotateX(50deg) rotateZ(-20deg) translateZ(calc(var(--layer-
index) * 36px));
}

.plane-label {
  position: absolute;
  top: 4px;
  right: 8px;
  font-size: var(--font-size-label);
  font-weight: var(--font-weight-bold);
  color: #aaa;
}

/* Only the front plane (layer 0) shows its label prominently */
.plane[data-layer="0"] .plane-label {
  color: #000;
  font-size: 1.2rem;
}
```

- In `app.js`, write `createPlaneStack(containerId, numLayers)` that builds the DOM:

```
function createPlaneStack(containerId, numLayers = 12) {
  const container = document.getElementById(containerId);
  container.innerHTML = '';
  container.className = 'plane-stack';
  for (let i = numLayers - 1; i >= 0; i--) {
    const plane = document.createElement('div');
    plane.className = 'plane';
    plane.dataset.layer = i;
    plane.style.setProperty('--layer-index', numLayers - 1 - i);
    plane.innerHTML = `<span class="plane-label">${i}</span>
```

```
                        <div class="plane-content" id="plane-content-${i}">
</div>`;
    container.appendChild(plane);
  }
}
```

Done when: Call `createPlaneStack('screen-trace', 12)` in the console → 12 overlapping planes appear in perspective, labeled 0–11, looking like your Figma mockup.

Reference: Your own mockup (the stacked planes in screen 3 and the attention screens).

Task P9.3: Build the dot renderer (trajectory) and heatmap renderer (attention) (~2 hrs)

What to do:

Dot renderer (for trajectory view):

```
// positions: [{tokenIdx, x, y, opacity}] — x, y are 0–1 normalized
function renderDots(layer, positions, tokenColors) {
  const content = document.getElementById(`plane-content-${layer}`);
  content.innerHTML = '';
  positions.forEach(({ tokenIdx, x, y, opacity }) => {
    const dot = document.createElement('div');
    dot.className = 'trajectory-dot';
    dot.style.left = `${x * 100}%`;
    dot.style.top = `${y * 100}%`;
    dot.style.background = tokenColors[tokenIdx];
    dot.style.opacity = opacity;
    content.appendChild(dot);
  });
}
```

Add to `style.css`:

```
.plane-content {
  position: absolute;
  inset: 0;
}

.trajectory-dot {
  position: absolute;
```

```
  width: 10px;
  height: 10px;
  border-radius: 50%;
  transform: translate(-50%, -50%);
}
```

Heatmap renderer (for attention view):

```
// matrix: n_tokens × n_tokens, each value 0–1
function renderHeatmap(layer, matrix) {
  const content = document.getElementById(`plane-content-${layer}`);
  content.innerHTML = '';
  const n = matrix.length;
  const grid = document.createElement('div');
  grid.className = 'heatmap-grid';
  grid.style.gridTemplateColumns = `repeat(${n}, 1fr)`;
  matrix.forEach(row => {
    row.forEach(val => {
      const cell = document.createElement('div');
      cell.className = 'heatmap-cell';
      cell.style.opacity = val;
      grid.appendChild(cell);
    });
  });
  content.appendChild(grid);
}
```

Add to `style.css`:

```
.heatmap-grid {
  display: grid;
  width: 100%;
  height: 100%;
}

.heatmap-cell {
  background: var(--color-attention-max);
}
```

Done when: Call `renderDots(0, [{tokenIdx: 0, x: 0.4, y: 0.6, opacity: 0.2}], ['blue'])` → a dot appears on plane 0. Call `renderHeatmap(0, matrix)` → a blue gradient grid appears on plane 0.

Week 9 Deliverables Summary

Who	Deliverable	Done when
Mario	`attention_matrix` key fixed in `app.py` ; BOS note in `API.md` ; HTML scaffold with screen switching; `/tokenize` wired to token chips with selection	Click Tokenize → see chips; select chips → Trace button appears
Steven	All 4 fetch functions with offline fallback; `/trace` wired → dots on planes; `/attention` wired → heatmaps (single head + all-layers modes)	Click Trace → see trajectory planes; switch to attention → see heatmaps change when head is changed
Polly	`tokens.css` with design tokens; stacked planes CSS component; dot renderer; heatmap renderer	Stack looks like the Figma mockup; dots and heatmaps render correctly on the planes
All	Full click-through works: input → tokenize → select token → trace → attention	Gabrielle can run through all screens without errors on Friday

---

Created: 2026-03-06
Sprint: Mar 9–13, 2026